User Guide

How to run the MVS NFS server day to day: starting and stopping it, driving it from the operator console, and telling it which datasets to export.

For getting the software onto MVS in the first place, see installation.md. For the internals of the config parser and its error handling, see readme_config.md.

---

1. Starting and stopping the server

1.1 Starting

NFSD runs as a started task. With the supplied procedure in a JES2 proclib:

```
S NFSD
```

The procedure (`jcl/nfsd.jcl`) runs two steps:

Step	Program	Purpose
`RESSOCK`	`RESSOCK`	Frees sockets left behind by a previous crash, for the ports listed in its `PARM` . Harmless when there is nothing to free.
`NFSD`	`NFSD`	The server itself.

The server needs no APF authorisation. It does need read access (and, for writable exports, write access) to the datasets it exports and to its configuration file.

1.2 Start parameters

The `NFSD` step passes:

```
PARM='-p 111 -m 2048 -n 2049 NFSDCONF'
```

Parameter	Meaning	Default if omitted
`-p <port>`	Portmapper (RPC bind) port	`111`
`-m <port>`	MOUNT protocol port	`20048`
`-n <port>`	NFS protocol port	`2049`
`-v`	Verbose startup	off
(last operand)	Configuration file — **required**	none

The final operand names the configuration. In the supplied procedure it is `NFSDCONF`, matching the `//NFSDCONF DD` statement in the same step, so changing which dataset holds your configuration is a JCL edit rather than a PARM edit:

```
//NFSDCONF  DD DISP=SHR,DSN=&CONFIG
```

The configuration dataset may be sequential or a PDS member, `RECFM=FB` or `VB`.

1.3 Confirming it started

Two console messages tell you the server is up and what it is serving:

```
NFSDM100I ... time offset ...
NFSDM220I Loaded 2 export(s), 5 dataset(s) from NFSDCONF
```

If the export or dataset count is not what you expected, the configuration was parsed differently than you intended — see section 4.5.

Note on startup log volume: the built-in level before the configuration is read is `DEBUG`, so a configuration with no `[Init]` section logs very verbosely. Putting `set loglvl info` as the first line of `[Init]` is the usual practice.

1.4 Stopping

```
P NFSD
```

`STOP` is the correct way to shut down. The server will:

1. Logs `NFSDM060I MVS STOP command received, shutting down`.
2. **Flushes every pending member write** — members buffered in the write pool are stowed into their PDS, so nothing in flight is lost.
3. Closes all client connections and the three listening sockets.
4. Writes the performance statistics (the same report as `STATS LIST`).
5. Logs `NFSDM190I Closing sockets` and `NFSDM200I Shutting down`.

1.5 When STOP does not work

`CANCEL` is a last resort, not a routine alternative, because it skips every step above: buffered member writes are lost, and sockets may be left allocated until the next `RESSOCK` run.

Two things worth knowing if you do have to cancel:

- **The log tail is lost.** Log output is buffered and written out only on close, so a cancelled task loses whatever had not yet been flushed. Console (WTO) messages are real-time and survive; the SYSOUT log may not. If you need to see what the server was doing at the moment of a problem, raise the console level with `SET WTOLVL` rather than the log level.
- **A task that ignores `P` may be deadlocked.** There is a known condition in which an abend leaves an internal lock held and the next file operation waits forever — the task then responds to nothing but `CANCEL`. It is documented in analysis_io_lock_hang.md. Cancelling with a dump gives abend code `S122`; leaving the task alone gets it cancelled by MVS after an hour in a wait, with `S522`.

---

2. Operator commands (MODIFY)

Commands are sent with the MVS `MODIFY` command, abbreviated `F`:

```
F NFSD,<command>
```

Command text is case-insensitive. A command the server does not recognise is reported and ignored — it never stops the server:

```
NFSDM070W MVS MODIFY ignored (unrecognised): <text>
```

The same commands can be placed in the configuration file's `[Init]` section to run at startup; see section 4.1.

2.1 Command summary

Command	Effect
`SET LOGLVL <level>`	Set the global log-stream level
`SET LOGLVL <level> PROC=<name>`	Set the level for one NFS procedure only
`SET WTOLVL <level>`	Set the operator console (WTO) level
`STATS LIST`	Report performance statistics
`STATS RESET`	Zero the performance statistics

2.2 Logging levels

`<level>` is one of, from most detail to least:

`DEBUG` , `TRACE` , `INFO` , `WARN` , `ERROR` , `FATAL`

There are two independent thresholds:

- **`SET LOGLVL`** controls the log stream (the `STDERR` SYSOUT dataset).
- **`SET WTOLVL`** controls what also reaches the operator console. Because console output is real-time while the log stream is buffered, `SET WTOLVL` is the one to reach for when you are watching a problem happen.

Examples:

```
F NFSD,SET LOGLVL INFO
F NFSD,SET WTOLVL WARN
```

2.3 Per-procedure logging

Debugging one operation without drowning in the rest is what `PROC=` is for. It pins a single NFS procedure to its own level, independent of the global setting:

```
F NFSD,SET LOGLVL INFO
F NFSD,SET LOGLVL DEBUG PROC=WRITE
```

Valid procedure names:

`GETATTR`, `SETATTR`, `LOOKUP`, `ACCESS`, `READ`, `WRITE`, `CREATE`, `REMOVE`, `RENAME`, `READDIR`, `READDIRPLUS` (alias `RDIRPLUS`), `FSSTAT`, `FSINFO`, `PATHCONF`, `COMMIT`, `NULL`

A common trap: if the procedure you are investigating is not at `DEBUG`, its diagnostic messages simply will not appear, and the log's silence looks like the operation never happened. Set the procedure level *before* reproducing the problem.

2.4 Performance statistics

```
F NFSD,STATS RESET
   ... exercise the server ...
F NFSD,STATS LIST
```

`STATS LIST` reports, per NFS procedure, the call count and the total, average, minimum and maximum elapsed time in milliseconds. The same report is produced automatically at shutdown.

This is the tool for answering "why is this slow?". Compare the *average in-server time* against the round-trip time seen on the wire: if a procedure averages a couple of milliseconds but each round trip costs far more, the time is going to the network and to the client's request count, not to the server, and tuning the server will not help.

Both commands temporarily raise the log level so their output always appears, then restore your previous setting.

---

3. The configuration file

One file configures the server. It is named as the last start parameter (section 1.2) and is read once at startup — changing it requires a restart.

---

4. Configuration sections

The file is sectioned, in Windows `.ini` style:

```
[Init]
set loglvl info
set loglvl debug proc=write

[Exports]
/exports    TEMP.TESTPROJ.C
/exports    TEMP.TESTPROJ.CNTL
```

General rules:

- A `[Name]` line switches section. Section names are case-insensitive, and blanks inside the brackets (`[ Init ]`) are accepted.
- Lines beginning with `#` are comments; blank lines are ignored. Both are allowed anywhere.
- Lines are processed **in file order**, so `[Init]` commands take effect before any section written below them.
- Lines appearing before any section header are treated as `[Exports]` .
- An unrecognised section is reported and skipped, so a configuration written for a newer server still loads on an older one.

4.1 `[Init]` — startup commands

Every line here is an operator command, run at startup exactly as if it had been typed at the console. Write the text that would follow `F NFSD,` :

```
[Init]
set loglvl info
set loglvl debug proc=write
set wtolvl warn
```

The available commands are those in section 2. They are handled by the same code as the console path, so the two cannot drift apart.

4.2 `[Exports]` — the datasets you are serving

One PDS per line:

```
<nfs-export-path>    <pds-dataset-name>
```

Repeating the **same export path** groups several datasets under one export. Each dataset appears as a directory named the **lower-case form of its dataset name**, with its members as the files inside:

```
[Exports]
/exports    TEMP.TESTPROJ.C
/exports    TEMP.TESTPROJ.CNTL
```

gives clients:

```
/exports/                       (export root -- a virtual directory)
  temp.testproj.c/              (PDS TEMP.TESTPROJ.C)
    prog1.c
    prog2.c
  temp.testproj.cntl/           (PDS TEMP.TESTPROJ.CNTL)
    job1.cntl
```

File extensions. Members are shown with an extension derived from the dataset's last qualifier, lower-cased: members of `TEMP.TESTPROJ.CNTL` appear as `*.cntl`. This is not cosmetic — a filename whose extension does not match is not a valid member name in that PDS and will be rejected. Use `fileext=` or `nofileext` (section 4.3) to change it.

Limits. Up to 16 export paths, each holding up to 32 datasets.

4.3 Export keywords

Keywords tune read-only status, the permissions reported to clients, and the file extension. They are case-insensitive and order-independent.

Keyword	Level	Meaning	Default
`ro`	export, dataset	Refuse all mutating operations	*(read-write)*
`rw`	export, dataset	Explicitly read-write	*(on)*
`dirperm=<octal>`	export, dataset	Mode reported for a PDS directory	`777`
`memperm=<octal>`	export, dataset	Mode reported for its members	`777`

Keyword	Level	Meaning	Default
`rootperm=<octal>`	**export only**	Mode reported for the export root	`555`
`fileext=<ext>`	export, dataset	Extension shown for members, overriding the derived one	*(last qualifier)*
`nofileext`	export, dataset	Show members with no extension at all	*(off)*

Permission values are **octal** (`755` means `0755`), in the range `0` – `777` . A leading zero is accepted. There are no setuid, setgid or sticky bits.

`fileext=` takes a bare extension with no dot (`fileext=jcl` shows `member.jcl`), 1–15 characters, stored lower-cased. It is useful when the last qualifier is not the extension you want clients to see — presenting a `.CNTL` PDS as `*.jcl` so an editor recognises it, for instance.

`nofileext` suppresses the extension entirely, so members appear under their bare name (`iefbr14` rather than `iefbr14.samplib`). `fileext=` and `nofileext` cannot both appear on one line. With no extension in effect, a filename containing a dot is not a valid member name and is rejected.

`ro` is enforced, not merely advertised: the write bits are stripped from the reported mode so `ls -l` and the NFS `ACCESS` check agree with reality, and the enforcement is independent of the client's user id — a client mounting as root still cannot write. A refused write returns `NFS3ERR_ROFS` .

The **export root is always read-only** — it cannot be modified through NFS, since MKDIR and RMDIR are not supported. `rootperm=` only tunes its read and execute bits.

4.4 Two ways to write it

Flat form — keywords follow the dataset name and apply to that dataset:

```
[Exports]
/pub      SYS1.PROCLIB                    ro
/exports  TEMP.TESTPROJ.C                 dirperm=755 memperm=644
/exports  TEMP.TESTPROJ.CNTL   fileext=jcl dirperm=777 memperm=666
/exports  SYS2.JCLLIB          fileext=jcl ro
/exports  SYS1.SAMPLIB         nofileext   ro
```

One mount can mix read-only and read-write datasets this way, and give each its own extension or none.

Block form — to set an option once for a whole export, or to set `rootperm` (which is export-level only), use a brace block. Keywords before the `{` are inherited by every dataset inside; keywords after a dataset name refine them:

```
[Exports]
/pub  ro rootperm=555 dirperm=555 memperm=444 {
    SYS1.PROCLIB
    SYS1.MACLIB     memperm=400
}
```

Inheritance rules:

- `dirperm`, `memperm` and the extension are **defaults** that a dataset may override.
- `ro` is a **ceiling**. A dataset may narrow to `ro` inside a read-write export, but `rw` on a dataset inside an `ro` export is an error.

4.5 A bad keyword drops the whole export

An unrecognised *section* is warned about and skipped, but a bad **export** keyword — an unknown keyword, an unparseable or out-of-range value, `rootperm` on a dataset, `rw` inside an `ro` export, or a malformed block — **discards that entire export**, with an error logged.

This is deliberate. The risk is asymmetric: an unknown section cannot make data writable, but a mistyped keyword can. A configuration saying `read-only` instead of `ro` must not silently leave an export writable. A missing export is safe and obvious; a partially applied one is neither.

That is why the `NFSDM220I` export and dataset counts at startup (section 1.3) are worth reading.

---

5. Mounting from a client

Mount the **export path** exactly as written in `[Exports]`.

Linux

```
sudo mount -t nfs -o vers=3,proto=tcp,nolock 192.168.1.168:/exports /mnt/mvs
```

Windows (Services for NFS)

```
mount -o anon,nolock \\192.168.1.168\exports X:
```

macOS

```
sudo mount -t nfs -o
vers=3,tcp,nolock,nobrowse,acregmin=600,acregmax=3600,acdirmin=600,acdirmax=
3600 192.168.1.168:/exports /Volumes/mvs
```

Two macOS-specific notes, both measured rather than assumed:

- `nobrowse` **is worth having.** Without it the macOS menu bar process polls the mount continuously — several RPCs per second, indefinitely, even when nothing is using it. Harmless in itself, but it competes with real work on a single-threaded server.
- `ls -l` **is dramatically slower than** `ls`**.** On a 1400-member library, `ls` takes about 6 seconds and `ls -l` about four minutes. `ls -l` displays the extended-attribute flag, and since NFSv3 has no extended attributes macOS probes for an AppleDouble `._` sidecar file for every entry — two extra round trips per member. Nothing on the server can prevent this. The attribute-cache settings above at least make a *repeat* `ls -l` fast; raise or lower them according to how quickly changes made on MVS need to become visible on the Mac.

---

6. Further reading

The full development tree is available on *github* at -- https://github.com/twinslow/mvs_nfsd

Document	Covers
installation.md	Getting the software onto MVS
readme_config.md	The configuration parser in detail, including EBCDIC notes
design_export_options.md	Why the export options behave as they do
limits_and_tunables.md	Every compiled-in limit and what happens at it
analysis_io_lock_hang.md	The known hang condition and its diagnosis